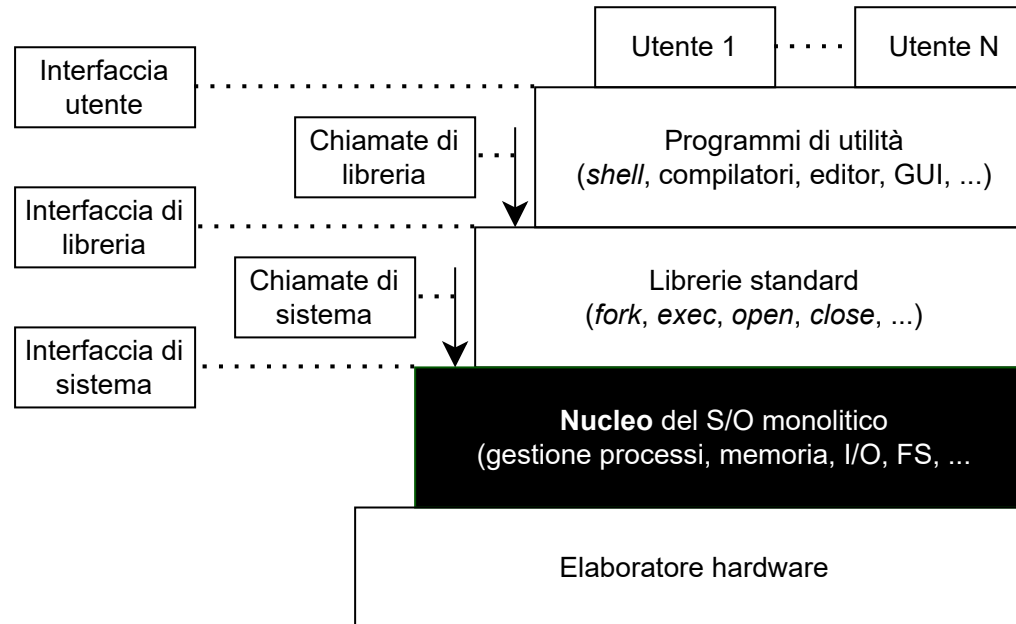


Linux

Sistema **multi-programmato** e **multi-utente**.

L'architettura è a *livelli gerarchici*.



UNIX nasce con una interfaccia utente per linea di comando (*shell*), è quindi più potente e flessibile di una **GUI** ma per utenti esperti. Si ha una *shell* per ogni terminale utente (*xterm*):

- caratteri di *prompt* (% , \$): indicano dove modificare il comando
- *pipe*: comando composto che prende l'output di comandi e l'utilizza come input di altri (tramite |)
- i comandi possono essere inviati all'esecuzione liberando la *shell* (& , *background*)

Gestione dei Processi

Un **Processo** è la principale entità attiva nel sistema, inizialmente *sequenziale*. Si verifica della concorrenza tra processi: molti processi vengono attivati dal sistema (*daemon*). I processi figli hanno memoria identica a quella del processo genitore (all'inizio), poi alla prima modifica diventa *indipendente*.

Thread

Processi con più flussi di controllo interni sono detti **thread**. I *thread* condividono tutte le risorse logiche e fisiche del processo genitore (inclusi i valori delle variabili). Una volta che completa il proprio lavoro il *thread* termina se stesso volontariamente. Un *thread* si può sincronizzare con la terminazione di un suo simile, l'accesso a risorse condivise si sincronizza tramite **semafori** a *mutua esclusione*. L'attesa su condizioni logiche avviene con variabili speciali simili a *condition variables* (ma senza monitor).

Permanentemente in RAM e per tutti i processi ci sono:

- Parametri di ordinamento (priorità, tempo di esecuzione, tempo di sospensione, ...)
- Descrittore della memoria virtuale del processo
- Lista dei segnali significativi e il loro stato
- Stato, identità, relazioni di parentela, gruppo di appartenenza

```

1  while(True){
2      type_prompt(); // mostra il prompt sullo schermo
3      read_command(cmd, par); // legge linea di comando
4      pid = fork();
5      if(pid < 0){
6          printf("Errore di attivazione del processo");
7          continue; // si ripete il ciclo
8      };
9      if(pid != 0){
10         // codice eseguito dal padre
11         waitpid(-1, &status, 0); // attende la terminazione di un figlio
12     }else{
13         // codice eseguito dal figlio
14         execve(cmd, par, 0);
15     };
16 };

```

All'istruzione `pid = fork()` il processo che chiama al Kernel e inserisce i dati per il figlio nella **tabella dei processi**. Se ci riesce viene allocata la memoria per lo *stack* e i dati del figlio. A questo punto il codice del figlio è lo stesso del padre.

Con il comando `execve(cmd, par, 0)` la linea di comando emessa dall'utente viene passata al processo figlio come *array* di stringhe. La *exec*, che opera nel Kernel, trova il programma da eseguire e lo sostituisce al codice del chiamante.

fork(): duplica il processo chiamante creando un figlio uguale al padre ma distinto.

Il **multi-threading** aggiunge complessità al *FS* e alla comunicazione tra entità attive.

chiamata di sistema (*system call*) alternativa a **fork()**:

```
1 pid = clone(function, stack_ptr, ctrl, arg);
```

- `function` = programma da eseguire nel nuovo processo o thread con argomento `par`
- `stack_ptr` = indirizzo dello stack assegnato al processo o thread
- `ctrl` = grado di condivisione desiderato tra il processo e l'ambiente del chiamante: spazio di memoria, FS, file, segnali, identità

I **thread** sono gestiti direttamente dal *Kernel*:

- ordinamento per *task* (thread o processo)
- selezione distinta
- prerilascio per fine quanto o per attesa

Ci sono 3 classi di priorità di *task*:

- Tempo reale politica FCFS a priorità senza prerilascio
- Tempo reale con politica RR a priorità
- Divisione di tempo RR a priorità

Con versione < 2.6 del *Kernel* i processi attivi non avevano prerilascio. Dalla versione 2.6 in poi si ha inibizione selettiva di prerilascio, si usano i semafori e si minimizza l'uso della disabilitazione delle interruzioni.

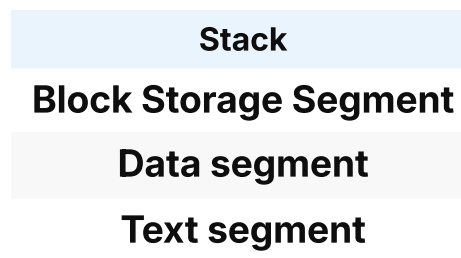
Inizializzazione

Il BIOS carica l'MBR sul disco di boot in RAM, l'MBR carica il programma di *boot* dal corrispondente blocco della partizione attiva

- **Processo 0:** configurazione clock, installazione del FS, creazione di processi 1(*init*) e 2(*daemon*)
- **Processo 1:** configurazione della modalità utente(singolo o multi), esecuzione *script* di inizializzazione **shell**, lettura del numero e tipo di terminali, `fork` per ogni terminale abilitato
- **Processo getty:** configurazione terminale e attivazione prompt di login, verifica password login e `exec("shell")`.

Gestione Memoria

Ha una semplicità massima e massima portabilità su diverse architetture. Ogni *processo* ha un proprio spazio di indirizzamento privato (**memoria virtuale**). Diviso in:



- Il *segmento dati* varia in dimensione in base alle attività del programma.
- Lo *stack* contiene l'ambiente d'esecuzione attuale (*record* di attivazione) e cresce in direzione opposta al *segmento dati*.

- Il **segmento codice** può essere condiviso con più processi, ma non gli altri segmenti per evitare la **duplicazione** di `fork()`.

Inizialmente l'allocazione di memoria principale era tramite **swap** di processi. Il gestore (**swapper**) creava lo spazio necessario salvando su disco i processi sospesi con più tempo di esecuzione recente e priorità minore.

Successivamente, fu introdotta paginazione **a richiesta** (**paging on demand**): un processo è eseguibile se il suo descrittore e la sua tabella delle pagine si trovano in RAM.

Page daemon controlla periodicamente se in RAM ci sono tot pagine libere. Se c'è spazio libero il **daemon** riporta in RAM i processi pronti, caricando solo il descritto di processo e la sua tabella delle pagine.

Per architetture a 32 bit la **memoria virtuale** di processo è di 4GB. Di cui 1GB è riservato e invisibile per la tabella delle pagine e per dei dati di controllo usati dal **Kernel**. Lo spazio è diviso in **regioni (sequenze)** contigue di pagine, ogni **regione** ha un descrittore.

La `fork()` replica per il **figlio** la lista di descrittori del **padre**. Le pagine del **figlio** sono duplicate **solo** nella modifica (**copy on write**).

Il **Kernel** rimane **sempre** in RAM. Nella RAM rimanente possono trovarsi:

- Le pagine attive dei processi utente
- Un insieme di pagine utente inattive ma presenti
- Una **cache** di blocchi di **file** usata dal FS

La RAM è allocata dinamicamente e in modo variabile.

Gestione I/O

UNIX tratta i dispositivi di I/O come *file* speciali, con una posizione specifica nel FS ad esempio `/dev/...`. Un **gestore** (*device driver*) è associato in modo esclusivo a ciascun dispositivo o famiglia di dispositivi dello stesso tipo.

GNU/Linux consente invece caricamento **dinamico** dei moduli di gestione dei dispositivi. Il caricamento dinamico richiede al **Kernel** di fare diverse azioni di configurazione:

- Rilocazione dello spazio di indirizzamento del modulo
- Allocazione risorse necessarie
- Configurazione vettore delle interruzioni
- Attivazione e inizializzazione del gestore

Un *file* speciale, **socket**, viene utilizzato per la connessione di rete e i protocolli. Un **socket** è associato a uno specifico indirizzo di rete. Ci sono tre tipi di connessione:

- Connessione **affidabile** a flusso di caratteri (**TCP**): il gestore garantisce la correttezza, invio e ricezione per blocchi variabili
- Connessione **affidabile** a flusso di pacchetti (**TCP**): invio e ricezione per pacchetti
- Trasmissione **inaffidabile** di pacchetti (**UDP**): utente deve gestire gli errori

File System

UNIX

Il *file* è visto dal FS come una sequenza di byte. Ci sono: *file* regolari, *file* repertorio (*directory*) e *file* speciali per i dispositivi I/O.

Il *file* è designato da un **cammino** assoluto o relativo.

- Il cammino relativo richiede di conoscere la *directory* di lavoro corrente

Tramite `mount`, Un FS B che si trova su una partizione visibile, può essere ritenuto parte di un FS A . Si può accedere a B tramite A partendo da un cammino designato, detto *mount point*.

Il **Kernel** usa due strutture di controllo:

- Un insieme di **tabelle di processo** contengono *descrittori utente* dei *file* in uso dai processi
- Una **tabella globale** mantiene la corrispondenza tra tutti i *file* aperti e i loro *i-node*
Ogni voce della **tabella di processo** punta a una voce della **tabella globale** che specifica **diritti** e **posizione di R/W** attuale nel *file*.

Sono previsti fino a 3 livelli di *indirizzazione* per gli *i-node* (64B) e 12 indirizzi diretti dell'*i-node* principale.

GNU/Linux

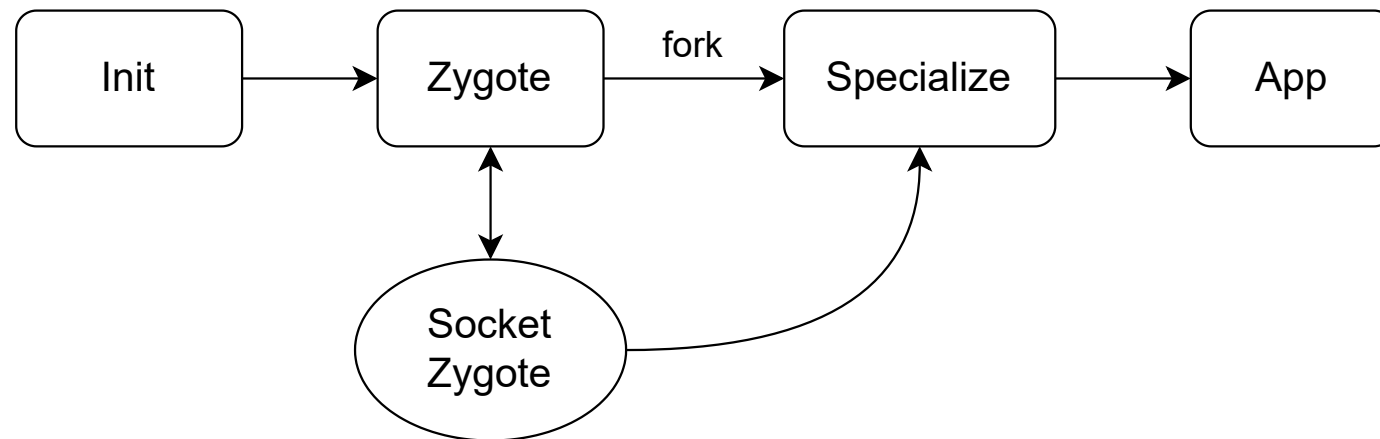
Come FS viene scelto **ext2**, che suddivide la partizione in **gruppi di blocchi**. Si ha quindi una distribuzione uniforme delle **directory** su disco, gli **i-node** e i relativi blocchi dati sono vicini tramite la preallocazione di alcuni blocchi quando si crea un **file**.

Gli **i-node** hanno una dimensione estesa a 128B, indirizzi di blocco ampi 4B. La scelta della dimensione dei blocchi (1,2,4 KB) è scelta in configurazione del FS.

Ci sono 12 indirizzi **diretti** e 3 **indiretti** (fino a terzo livello di indirezione).

Android

Sistema **open-source** personalizzabile per vari dispositivi.



- **Init** è il primo processo utente, come su **Linux**, che avvia processi a basso livello tra cui **zygote**
- **Zygote** avvia altri processi, tra cui **system_server**, ma anche altri
- **App** interagiscono con il SO con un set di chiamate a librerie, detto **Android Framework**

- *App* comunicano tra loro e con i servizi forniti dal SO (*system_server*) tramite *IPC Binder*
- *Wake Lock* sistema per inibire lo spegnimento e la gestione energetica
- *Out-of-memory-killer*: viene usata per chiudere i processi quando **Android** non ha abbastanza spazio
- *Android RunTime*: implementa un ambiente Java e ogni applicazione usa il proprio *ART*

I processi delle app non escono mai in modo pulito, ci pensa il **Kernel**.